Assignment #4: Spark

Introduction

You have been hired as a Data Engineer for the NYC Taxi & Limousine Commission (TLC). The strategy team wants to understand driver efficiency, passenger tipping behavior, and supply/demand imbalances across different boroughs.

Currently, the analysts are trying to load gigabytes of trip data into Pandas, resulting in constant `MemoryError` crashes. Your goal is to build a scalable, distributed batch processing pipeline using Apache Spark, orchestrate it with Airflow, and load the final aggregates into Snowflake.

This assignment focuses on core Data Engineering competencies:

- **Distributed Data Manipulation:** Using PySpark DataFrames to clean, join, and aggregate large datasets.
- **Advanced Analytics:** Utilizing Window functions for rank and time-series analysis.
- **Orchestration & Data Warehousing:** Automating workflows with Airflow and loading data into Snowflake.
- **Performance Optimization:** Understanding Spark's lazy evaluation, caching, and partitioning.

Resources & Documentation

Crucial Note: A primary objective of this assignment is to practice reading and applying technical documentation. You are expected to consult the Apache Spark PySpark Documentation to understand how to properly implement the required functions (e.g., `pyspark.sql.functions`, `pyspark.sql.Window`), as well as Airflow and Snowflake connector docs.

Part 0: Environment & Data Setup

For this assignment, we will be using the school-managed LinuxLab servers to ensure everyone has access to a consistent, high-memory environment capable of running Spark efficiently.

1. **Connect to LinuxLab:**
    - Please review the official class LinuxLab setup guide for full details.
    - *Summary:* You will need to SSH into `shell.engr.wustl.edu` using your student credentials. We highly recommend setting up VS Code Remote - SSH or launching a Jupyter Notebook session via the server. This allows you to edit files,

view notebooks, and utilize the server's compute power exactly as if they were on your local machine.
    - Once connected, follow the guide to activate the appropriate Python virtual environment containing PySpark, Airflow, and the Snowflake connectors.
2. **Download the Data:**
    - Once on the LinuxLab server, run the provided `download_data.py` script from your terminal: `python download_data.py`.
    - This script will download a full year of official Yellow Taxi Trip Records (Parquet format) for 2023 (approx. 500MB total), as well as the Taxi Zone Lookup table (CSV format).

Part 0.5: Tips for Running Spark on LinuxLab

1. **Running your code:** You can develop your assignment interactively in a Jupyter Notebook on LinuxLab, or execute it as a Python script (`python spark_taxi_student.py`). You do not need to use the `spark-submit` command while testing locally. Instead, your starter code is configured to read the `SLURMD_NODENAME` and `SPARK_MASTER_PORT` environment variables provided by your LinuxLab session. This connects your script directly to the Spark master node provisioned for your server.
2. **Viewing the Spark UI:** * Spark provides a fantastic Web UI to visualize execution DAGs, job progress, and memory usage. By default, it hosts this UI on port `4040`.
    - Because port forwarding can sometimes be unreliable across SSH, we recommend using the Spark UI directly on the **LinuxLab virtual desktop**, just as we did in class. Open a web browser within your LinuxLab VNC/FastX session and navigate to `http://localhost:4040` while your Spark job is running.
    - *Pro-Tip:* The Spark UI is only available while the Spark application is actively running. If your script finishes, the UI shuts down. If you want time to explore the UI, add `import time; time.sleep(600)` to the very end of your code block to keep it alive for 10 minutes after it finishes calculating!
3. **Navigating the Console Logs:** Spark runs on the JVM (Java Virtual Machine) and outputs a massive amount of `INFO` logging to the console. This is totally normal. Your `.show()` outputs will be buried in there. If your code crashes, scroll up through the logs and look specifically for lines starting with `ERROR` or the standard Python `Traceback` to find the root cause.

⚠️ Warning: Real-World Data is Messy! (Schema Drift)

Unlike perfectly curated textbook datasets, the NYC Taxi dataset is a real, operational dataset. Over the course of 2023, the NYC TLC made internal changes to how they record their data, leading to **type mismatches** across the monthly files.

For example, in early 2023, `passenger_count` was stored as a `BIGINT` (Long), but in later months, it was stored as a `DOUBLE`. If you try to load the entire year at once using a wildcard (e.g., `spark.read.parquet("data/*.parquet")`), Spark will likely crash with a cryptic error like:

```
java.lang.ClassCastException: class
org.apache.spark.sql.catalyst.expressions.MutableDouble
cannot be cast to class
org.apache.spark.sql.catalyst.expressions.MutableLong
```

💡 Pro-Tips for Working Through This:

1. **Start Small:** Don't try to process the entire year right away! Build, test, and debug your pipeline using just **one month** (e.g., `yellow_tripdata_2023-01.parquet`). Once your logic works perfectly for one month, scale up to the full year.
2. **Align the Schemas (The "Union" Trick):** To safely load the entire year and bypass the type mismatches, you should:
    - Read the monthly files individually (e.g., in a `for` loop).
    - Use PySpark's `.cast()` function to force conflicting columns into a unified data type (e.g., cast `passenger_count`, `RatecodeID`, and `airport_fee` to `DoubleType` for every month as you read it in).
    - Stitch the DataFrames together iteratively using `df1.unionByName(df2, allowMissingColumns=True)`.

Part 1: Data Cleaning & Preprocessing (Estimated Time: 30-45 mins)

Raw data is notoriously messy. Implement the `clean_taxi_data` function in your starter code to perform the following:

1. Filter out trips where the `passenger_count` is 0 or null.
2. Filter out trips where the `trip_distance` is less than or equal to 0.
3. Filter out trips where the `fare_amount` is less than or equal to 0.
4. Create a new column called `trip_duration_minutes` calculating the time difference between `tpep_dropoff_datetime` and `tpep_pickup_datetime` in minutes. Filter out any trips that are less than 1 minute or greater than 120 minutes.

Part 2: Core Analysis & Joins (Estimated Time: 45-60 mins)

The data currently uses `PULocationID` (Pickup Location ID) and `DOLocationID` (Dropoff Location ID).

1. Implement `join_zone_lookups` to join the cleaned trip data with the Zone Lookup DataFrame so you have the actual Borough and Zone names for both pickup and dropoff. *Hint:* You will need to join the lookup table twice (using aliases).
2. Implement `calculate_busiest_boroughs` to find the total number of trips originating from each Pickup Borough.

Part 3: Advanced Analytics (Window Functions) (Estimated Time: 1-1.5 hours)

The dispatch team wants to know the top 3 dropoff zones for each pickup borough based on trip volume.

1. Implement `calculate_top_dropoff_zones_by_pickup`.
2. Group the data by Pickup_Borough and Dropoff_Zone, counting the number of trips.
3. Use a PySpark **Window** function partitioned by Pickup_Borough and ordered by the trip count (descending) to assign a rank.
4. Filter to keep only the top 3 dropoff zones for each pickup borough.

Part 4: Orchestration with Airflow (Estimated Time: 45-60 mins)

In a real-world scenario, you wouldn't execute these scripts manually. Data pipelines must be orchestrated. Create an Apache Airflow DAG file (e.g., `taxi_pipeline_dag.py`) that automates this workflow.

A Note on Airflow and Spark: In a standard production environment, you would use Airflow's `SparkSubmitOperator` to run this job. The `SparkSubmitOperator` is preferred because it natively interacts with Spark cluster managers (like YARN or Kubernetes), handles Spark-specific configuration arguments cleanly, and tracks the distributed application's state directly.

However, because of our specific LinuxLab server setup and how we are managing local Spark environments via environment variables, the `SparkSubmitOperator` won't connect properly. Therefore, you must use a `BashOperator` to execute your script directly.

Why a BashOperator instead of a PythonOperator? While you *could* theoretically import your Python function into your DAG and run it via a `PythonOperator`, it is highly discouraged for Spark jobs. The `PythonOperator` executes code directly within the Airflow worker's memory

space. Because Spark jobs are extremely memory-intensive, running them inside the Airflow worker can easily cause Out-Of-Memory (OOM) errors, crashing your entire orchestration tool.

By using a `BashOperator` (e.g., executing the shell command `python spark_taxi_student.py`), Airflow spawns a completely separate OS process for your Spark job. This isolates the heavy data-processing workload from Airflow's internal processes and keeps your environment stable.

Your DAG must include at least two linked tasks:

1. A `BashOperator` task that executes the `download_data.py` script to ensure data is present.
2. A `BashOperator` task that runs your `spark_taxi_student.py` job for processing.

Ensure your tasks are set up sequentially (download -> process) so that Spark doesn't attempt to run before the data is available.

Part 5: Loading Results to Snowflake (Estimated Time: 30-45 mins)

Once your Spark job has processed and aggregated the data, the final step in our pipeline is to load these results into a data warehouse for the analytics team. Modify your Spark script to write the final aggregated DataFrames (from Part 2 and Part 3) directly to your Snowflake account. You will need to use the Spark-Snowflake connector. *Note: Ensure you are using environment variables or a secure connection method to pass your credentials—do not hardcode your Snowflake password in your script!*

Part 6: Extra Credit (Choose 1 or come up with your own!)

Go beyond the base requirements. Implement your logic in the `extra_credit` function. Some ideas:

- **Handling Data Skew:** Manhattan has vastly more trips than Staten Island. Simulate a skewed join scenario and implement a "Salting" technique to distribute the join evenly across executors.
- **Driver Sessionization:** Assuming `VendorID` represents a single driver (for the sake of this exercise), use Window functions to calculate the time elapsed between the dropoff of one trip and the pickup of the next trip for the same vendor.
- **Weather Integration:** Find a free historical weather API/CSV for NYC in 2023. Join this with the taxi data and calculate the average tip percentage on rainy days vs. clear days.

Part 7: Analysis and Reflection Questions

Please answer the following questions in a PDF document.

1. **Transformations vs. Actions:** What is the difference between a transformation and an action in Spark? List two transformations and one action you used in your code. What is "lazy evaluation"?
2. **Shuffling:** What is a Spark shuffle? Why is it considered an expensive operation? Which specific operation in Part 3 caused a shuffle?
3. **Caching:** Explain the difference between `cache()` and `persist()`. Since you are running this pipeline on a full year of data (12 Parquet files), at which specific point in your code would it be most beneficial to call `cache()`, and why?
4. **Narrow vs. Wide Dependencies:** Explain the difference between narrow and wide dependencies. Is a filter operation narrow or wide? Is a groupBy narrow or wide?

Feedback

1. How long (in hours) did this assignment take you to complete?
2. Did you use an LLM AI assistant at all working on this assignment? If so, which did you use and what relevant queries did you ask? *(Note: please include the exact queries you used. I would like to be able to recreate what the LLM responded with.)*
3. On a scale of -10 (hated it) to 10 (loved it), how much did you enjoy this assignment?
4. On a scale of -10 (too small) to 10 (too large), with 0 being perfectly sized, how would you rate the size and scope of this assignment?
5. On a scale of -10 (not valuable at all) to 10 (extremely valuable), how valuable did you find this assignment? This includes both how much you learned and how good the practice was.
6. Do you have any other feedback?

Submission Instructions

- **Canvas:** Submit a generic zip file containing your python files (`spark_taxi_student.py`, `taxi_pipeline_dag.py`, and any additional scripts you created for extra credit).
- **Gradescope:** Submit a PDF containing your answers to Part 7 and the Feedback section.